1. 과제 개요

특징 추출 (feature extraction)에 대한 과제입니다. PCA와 LDA를 직접 구현해서 데이터의 차원을 줄이는 것이 목표입니다.

- PCA (Principal Component Analysis): 데이터 전체의 분산을 기준으로 차원 축소
- LDA (Linear Discriminant Analysis): 클래스 분류에 유리하도록 차원 축소

sklearn 등의 라이브러리를 사용하지 않고 numpy로 직접 구현해야 하는 조건이었습니다.

사용 환경: Python 3, NumPy, Matplotlib, h5py

2. 개념 정리

과제를 시작하기 전에 PCA와 LDA가 뭔지부터 찾아봤습니다.

PCA

데이터가 가장 넓게 퍼진 방향을 찾는 방법입니다. 순서는 다음과 같습니다.

1. 전체 데이터의 평균을 구해서 중심화
2. 공분산 행렬 계산
3. 고유값/고유벡터 구하기
4. 고유값이 큰 순서대로 주성분 선택

고유값, 고유벡터 개념이 처음에 잘 안 와닿았는데, 행렬을 곱해도 방향이 안 바뀌고 크기만 바뀌는 벡터가 고유벡터이고, 그 크기 변화량이 고유값이라고 이해했습니다.

LDA

PCA랑 다르게 클래스 정보 (레이블)를 사용합니다. 같은 클래스끼리는 뭉치고 (Sw 최소화), 다른 클래스끼리는 멀어지는 (Sb 최대화) 방향을 찾습니다.

Sw^(-1) * Sb의 고유벡터를 구하면 그게 최적의 분류 방향이 됩니다.

정리하면 PCA는 레이블 없이 분산 기준 (비지도), LDA는 레이블을 보고 분류 기준 (지도)으로 방향을 잡습니다.

3. 문제 1 - 인공 데이터에 대한 PCA와 LDA (총 40점)

3.1 문제 1-(1): 데이터 생성 및 산점도 (10점)

주어진 조건:

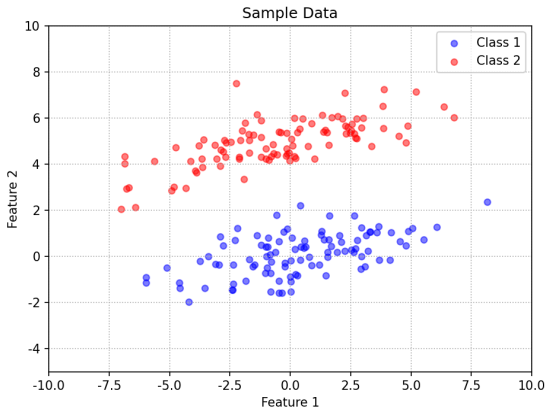
- 클래스 1: 평균 [0, 0], 공분산 [[10, 2], [2, 1]]
- 클래스 2: 평균 [0, 5], 공분산 [[10, 2], [2, 1]]
- 각 100개씩 총 200개

두 클래스의 중심이 y축으로 5만큼 떨어져 있고, x축 방향 분산(10)이 y축 방향 분산(1)보다 크니까 옆으로 길쭉한 형태가 될 거라고 예상했습니다.

np.random.multivariate_normal()으로 생성했습니다.

실행 결과:

- 클래스1 평균: [0.360, 0.103]
- 클래스2 평균: [-0.412, 4.946]



[특징 추출을 위한 샘플 데이터 생성]

산점도를 보니 예상대로 x축으로 길쭉하게 퍼져있고, y축 방향으로 두 클래스가 어느 정도 나뉘어 있었습니다. 근데 x축 방향으로는 많이 겹쳐있습니다.

3.2 문제 1-(2): PCA 및 LDA 벡터 시각화 (20점)

PCA와 LDA 각각이 찾은 방향을 화살표로 같은 그래프에 표시합니다.

PCA

공분산 행렬을 직접 계산했습니다. np.cov()를 쓸까 하다가, np.cov()는 1/(N-1)로 나누는 거라 수식이랑 안 맞아서 직접 (1/N) * X^T * X로 구했습니다. 고유값 분해는 eigh()를 사용했는데 이게 대칭행렬 전용이라 공분산 행렬에 맞습니다.

결과:

- 고유값: [9.424, 6.484]
- 첫번째 주성분 (PC1): [-0.944, -0.331]
- 분산 설명 비율: 59.2%

PC1이 x축 방향에 거의 평행합니다. 데이터가 x축으로 제일 넓게 퍼져있으니까 당연한 결과입니다.

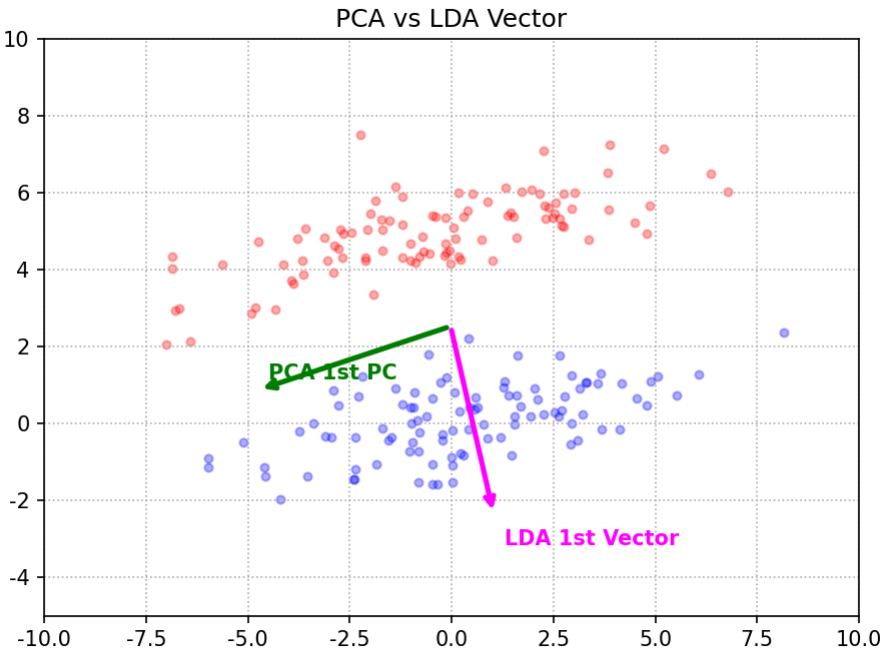
LDA

Within-class scatter(Sw)와 Between-class scatter(Sb)를 구하고, Sw^(-1) * Sb의 고유벡터를 구했습니다.

결과:

- 고유값: [11.252, 0.000]
- 첫번째 판별벡터: [0.212, -0.977]
- 2클래스라 유의미한 벡터는 1개뿐

LDA 벡터는 y축 방향에 거의 평행합니다. 두 클래스 평균이 y축으로 차이가 나니까 (mu1=[0,0], mu2=[0,5]) 이것도 맞는 결과입니다.

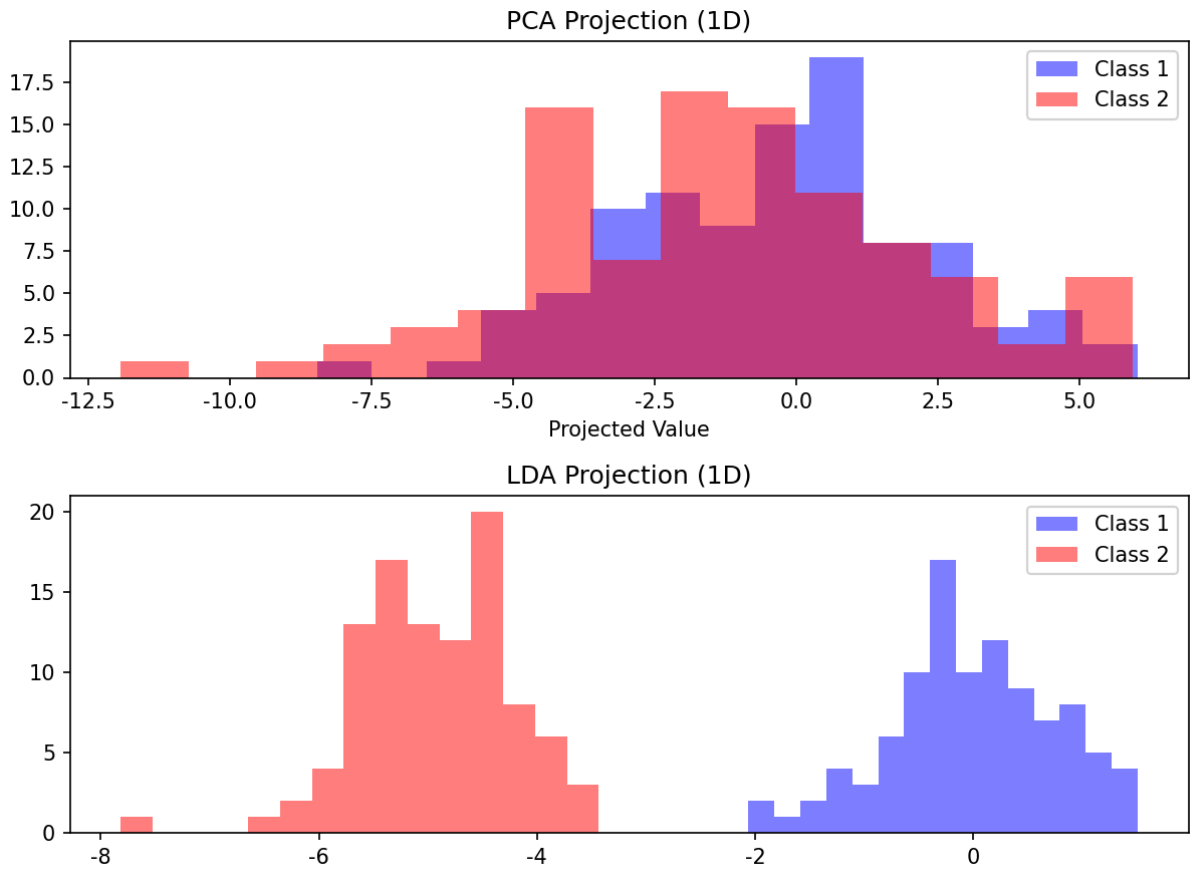


[그림 2. PCA 및 LDA 벡터]

그래프를 보면 PCA (초록)는 x축 방향, LDA (마젠타)는 y축 방향을 가리키고 있어서 거의 직교합니다.

3.3 문제 1-(3): PCA와 LDA 결과 비교 (10점)

각 방향으로 1차원 투영한 결과를 히스토그램으로 비교했습니다.



[PCA vs LDA 1차원 투영 비교]

PCA 투영: 데이터가 넓게 퍼져있긴 한데 두 클래스가 심하게 겹칩니다. PCA는 그냥 퍼짐이 큰 방향을 찾은 거라 분류랑은 상관없는 방향을 고른 셈입니다.

LDA 투영: 두 클래스가 확실히 분리됩니다. 처음부터 클래스를 나누는 걸 목표로 했으니까요.

구분	PCA	LDA
레이블 사용 여부	사용 안 함 (비지도)	사용함 (지도)
최적화 목표	데이터 전체의 분산 최대 보존	클래스 간 분리 최대화

찾은 방향	x축 방향 (분산이 큰 방향)	y축 방향 (클래스가 나뉘는 방향)
투영 후 분류 가능?	어려움 (겹침이 심함)	용이함 (잘 분리됨)

이 데이터에서는 분산이 큰 방향과 클래스가 나뉘는 방향이 다르기 때문에 PCA로는 분류가 안 되고 LDA가 필요합니다. 만약 두 방향이 같았으면 PCA도 잘 됐을 겁니다.

4. 문제 2 - COIL20 데이터에 대한 PCA와 LDA (총 60점)

4.1 데이터

COIL20은 20개 물체를 회전시키면서 찍은 이미지 데이터셋입니다. 학습 데이터는 280개 (20개 물체 * 14장), 각 이미지는 32 * 32 = 1,024차원입니다.

4.2 PCA 2차원 특징 추출

문제 1이랑 같은 방식으로 PCA를 적용했습니다. 다만 차원이 1,024라 행렬이 훨씬 큽니다.

정보보존율을 확인해봤는데
- 2차원으로 줄이면 약 43%밖에 안 남습니다
- 95%를 보존하려면 61개 주성분이 필요

1,024개 숫자 중에 상당수가 중복 정보라는 뜻입니다.

4.3 LDA 2차원 특징 추출 (PCA 95% 전처리 후)

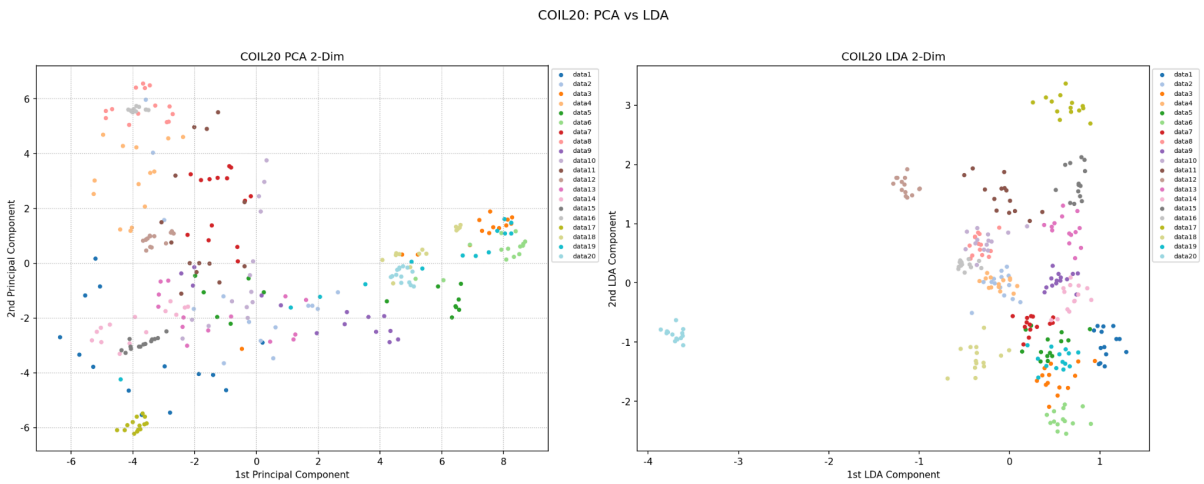
COIL20에 바로 LDA를 하려고 했더니 Sw 역행렬에서 에러가 났습니다. singular matrix라고 뜨는데, 찾아보니 Sw가 1024 * 1024인데 데이터가 280개밖에 없어서 rank가 부족한 게 원인이었습니다. (rank 최대 280-20=260인데 행렬 크기는 1,024)

그래서 과제 지시대로 PCA로 95% 보존되는 61차원까지 먼저 줄인 다음에 LDA를 적용했습니다. 이렇게 하면 Sw가 61 * 61이 되니까 역행렬을 구할 수 있습니다.

근데 역행렬 구할 때 또 불안정하다고 에러가 나서, Sw 대각선에 아주 작은 값(1e-6)을 더해주는 정규화를 적용했습니다. 이것도 검색해서 알게 됐습니다.

결과: 유의미한 판별벡터 19개 (클래스 수 - 1)

4.4 결과 비교



[COIL20 데이터 PCA 및 LDA 2차원 특징 추출 후 출력]

PCA (좌측): 클래스들이 전반적으로 겹쳐있습니다. 클래스 정보 없이 분산만 보고 줄인 거라 분류 관점에서는 별로입니다. 2차원으로는 43%만 보존되니 정보 손실도 큽니다.

LDA (우측): 같은 클래스끼리 더 뭉쳐있고 다른 클래스와의 분리가 확실히 나아졌습니다. 다만 20개 클래스를 2차원에 다 표현하기는 무리라 일부 겹치는 건 어쩔 수 없습니다.

문제 1의 인공 데이터에서 본 것과 같은 패턴이 실제 이미지 데이터에서도 나타났습니다.

5. 결론

1. PCA는 분산 보존, LDA는 분류 최적화 방향을 찾는다
2. 분류가 목적이면 LDA가 PCA보다 효과적이다
3. 고차원에서 LDA를 쓰려면 Sw의 특이행렬 문제가 있어서, PCA로 먼저 축소해야 한다

과제를 하면서 느낀 점은, 차원 축소가 단순히 숫자를 줄이는 게 아니라 어떤 기준으로 줄이냐가 중요하다는 것입니다. 그리고 PCA를 먼저 하고 LDA를 하라는 과제 지시가 단순한 절차가 아니라 수학적으로 필요한 과정이었다는 걸 이해했습니다.

아쉬운 점은 SVD를 이용한 PCA 구현이나, regularization epsilon 값을 어떻게 정하는 게 좋은지는 더 공부가 필요할 것 같습니다.